

AgentFlow — Complete Interview Preparation Guide

Goal of this document: make you able to explain *every part* of AgentFlow — from the simplest idea to the most advanced internals — in your own words, with confidence. Read it top to bottom once. Then re-read the **"Say it like this"** boxes until they feel natural. By the end you should be able to answer any question a recruiter throws at you about this project.

Table of Contents

1. The 30-second pitch
 2. The 2-minute walkthrough (memorize this)
 3. What problem does it solve?
 4. High-level architecture
 5. The tech stack and *why* each piece
 6. Monorepo (Turborepo) — why one repo
 7. Service 1 — Auth service
 8. Service 2 — Workflow service
 9. Service 3 — Execution Engine (the heart)
 - 9.1 Message queue (BullMQ + Redis)
 - 9.2 The worker
 - 9.3 Topological sort (running nodes in order)
 - 9.4 Node executors (Strategy pattern)
 - 9.5 The ReAct AI agent
 - 9.6 Multi-model LLM support
 - 9.7 Real-time updates (Socket.io)
 - 9.8 Execution logging
 10. The frontend
 11. End-to-end: what happens when you click "Run"
 12. Data models
 13. Security
 14. DevOps — Docker, CI/CD, monitoring
 15. Testing
 16. System design & scaling
 17. Trade-offs & "what would you improve?"
 18. Interview Q&A bank
 19. Glossary (plain-English definitions)
 20. One-page cheat sheet
-

1. The 30-second pitch

Say it like this: "AgentFlow is a visual AI workflow automation platform — think of it as an open-source mix of **n8n** and **Zapier**, but built for AI agents. You drag nodes onto a canvas — an HTTP call, an LLM prompt, an AI agent, a condition — connect them, and hit Run. The backend executes the whole pipeline asynchronously through a job queue and streams each node's status back to the screen in real time. It's built as **three microservices** with a **Redis-backed message queue**, **MongoDB**, and **Socket.io**, all in a TypeScript monorepo, containerized with Docker and deployed on the cloud."

Key numbers to drop: 3 microservices · 6 node types · 4 AI models · real-time over WebSockets · 29 tests · fully Dockerized · live in production.

2. The 2-minute walkthrough

Use this when they say "walk me through your project." Follow the order: **problem** → **what it does** → **how it's built** → **what's impressive** → **result**.

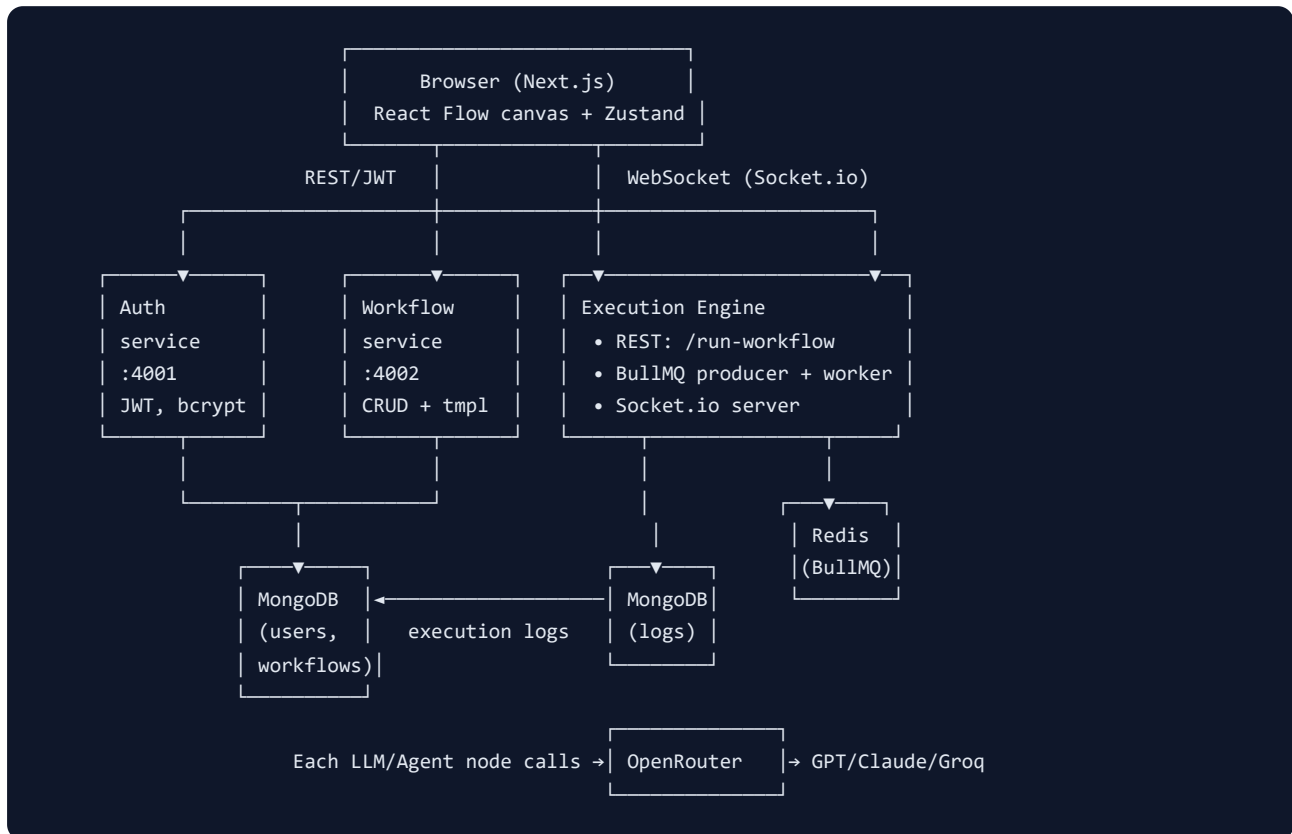
1. **Problem.** Teams want to automate tasks that involve AI — "fetch this data, summarize it with an LLM, decide something, send it on." Writing that as code each time is slow. Visual automation tools exist (Zapier, n8n) but aren't built around AI agents.
 2. **What it does.** AgentFlow gives you a drag-and-drop canvas. Each block ("node") is a step: HTTP fetch, LLM prompt, an autonomous AI agent, a condition branch, a webhook trigger, or a cron schedule. You wire them together into a directed graph and run it.
 3. **How it's built.** The frontend is **Next.js + React Flow** for the canvas. The backend is **three Express microservices**: Auth (JWT login), Workflow (save/load workflows), and the Execution Engine. When you run a workflow, the engine doesn't execute it inline — it pushes a job into a **BullMQ queue on Redis**. A worker picks it up, sorts the nodes by dependency using a **topological sort**, and runs each node. As each node finishes, it emits a **Socket.io** event so the canvas updates live. Everything persists to **MongoDB**.
 4. **What's impressive.** It's not a toy — it uses production patterns: async job processing with retries, real-time streaming, a pluggable node-executor system (Strategy pattern), a **ReAct AI agent** that can reason and call tools, support for **multiple LLMs** (GPT-4o, Claude, Groq) through one interface, plus Docker, CI/CD, and tests.
 5. **Result.** It's deployed and working — you can sign up, build a pipeline, and watch a real HTTP→LLM→Agent chain execute live against real AI models.
-

3. What problem does it solve?

- **Automation is repetitive to code.** Every "fetch → process → decide → act" task needs glue code. A visual builder lets non-experts (and experts, faster) compose them.
- **AI needs orchestration.** A single LLM call is rarely enough. Real tasks chain steps: get data, reason over it, branch on the result, call a tool. AgentFlow makes that chaining visual and re-runnable.
- **Observability.** When automation runs in the background you need to see what happened. AgentFlow streams live status and stores a full execution history.

Say it like this: "It turns multi-step AI automations from throwaway scripts into reusable, observable, visual workflows."

4. High-level architecture



Why this shape? Each box does one job and can be scaled or restarted independently. The queue (Redis) sits between "accept the request" and "do the heavy work" so the API stays fast and nothing is lost if a worker crashes.

5. The tech stack and *why* each piece

Layer	Technology	Why we chose it (say this)
Language	TypeScript	Type safety across the whole stack; shared types between frontend & backend.
Frontend	Next.js 14 (App Router)	React framework with routing, SSR, and easy deploys.
Canvas	React Flow	Purpose-built library for node-based drag-and-drop editors.
Client state	Zustand	Tiny, simple global store for nodes/edges/execution state.
Server state	React Query	Caching + fetching for workflows (loading, refetch, invalidation).
Backend	Node.js + Express	Lightweight, huge ecosystem, fast to build REST APIs.
Database	MongoDB + Mongoose	Flexible document model fits variable-shaped workflow JSON.
Queue	BullMQ + Redis	Durable, battle-tested job queue with retries & concurrency.
Real-time	Socket.io	Reliable WebSocket abstraction with rooms & reconnection.
AI	OpenRouter	One API key → many models (OpenAI, Anthropic, Groq).
Monorepo	Turborepo + npm workspaces	Share code, build in dependency order, cache builds.
Tests	Vitest	Fast, modern test runner with great TS support.
Containers	Docker (multi-stage)	Reproducible builds; same image runs anywhere.
CI/CD	GitHub Actions	Auto lint → test → build → docker on every push.
Monitoring	Prometheus + Grafana	Metrics scraping + dashboards.

Tip: Interviewers love "why did you pick X over Y." Have a one-liner for each, e.g. "*BullMQ over a plain `setTimeout` because jobs must survive a crash and retry — a queue gives durability, retries, and back-pressure for free.*"

6. Monorepo (Turborepo)

What: one Git repo holds everything — `apps/frontend`, three `services/*`, and shared `packages/*`.

```

agentflow/
├─ apps/frontend/           # Next.js UI
├─ services/
│   ├─ auth-service/       # login / signup (port 4001)
│   ├─ workflow-service/   # save/load workflows (port 4002)
│   └─ execution-engine/   # runs workflows (port 4003)
├─ packages/
│   ├─ shared-types/       # TypeScript interfaces used everywhere
│   └─ shared-utils/       # logger, custom error classes
├─ infrastructure/docker/   # one Dockerfile per service
├─ render.yaml             # cloud deploy blueprint
└─ turbo.json              # task pipeline

```

Why:

- **Shared types** — `WorkflowNode`, `AIModel`, etc. live in `packages/shared-types` and are imported by both the frontend and backend, so the contract can never drift.
- **One command** — `npm run build` builds packages → services → frontend in the right order, with caching (Turborepo).
- **Atomic changes** — a feature touching the API and the UI is one commit/PR.

Say it like this: "A monorepo means the frontend and backend literally share the same TypeScript interfaces, so if I change a workflow's shape, both sides get a compile error until they agree."

7. Service 1 — Auth service

Job: sign users up, log them in, and prove identity to the other services.

Endpoints: `POST /auth/signup`, `POST /auth/login`, `GET /auth/me`, `GET /health`.

How it works (step by step):

1. On **signup**, the password is hashed with `bcrypt` (12 salt rounds) — we never store the raw password.
2. We create the user in MongoDB and issue a **JWT** signed with a secret, expiring in 7 days.
3. On **login**, we look up the user, `bcrypt.compare()` the password, and re-issue a JWT.
4. Every protected request to other services sends `Authorization: Bearer <token>`; the service verifies the token with the same secret.

```

// signup (simplified)
const hashed = await bcrypt.hash(password, 12)
const user = await User.create({ email, password: hashed })
const token = jwt.sign({ id: user._id, email }, JWT_SECRET, { expiresIn: "7d" })
res.json({ success: true, data: { token, user: { id: user._id, email } } })

```

Hardening: `helmet` (secure HTTP headers), `cors` (controls who can call it), `express-rate-limit` (max 100 requests / 15 min to stop brute force), and a global error handler.

Likely Q: "Why JWT and not sessions?" JWTs are stateless — each service can verify a token with just the shared secret, no central session store needed. That fits a microservice setup where Auth and the Engine are separate processes."

Likely Q: "What's bcrypt doing?" "It's a slow, salted hashing function. Slow is good for passwords — it makes brute-force expensive. The salt means two identical passwords hash differently. 12 rounds is the work factor."

8. Service 2 — Workflow service

Job: store and serve users' saved workflows (the nodes + edges JSON) and provide starter templates.

Endpoints (all JWT-protected):

- `GET /workflows` — list my workflows
- `GET /workflows/:id` — one workflow
- `POST /workflows` — create
- `PUT /workflows/:id` — update
- `DELETE /workflows/:id` — delete
- `GET /workflows/meta/templates` — built-in templates

Model (Workflow): `userId, name, description, nodes[], edges[], isActive, tags[]` with timestamps.

Why separate from Auth? Different responsibility, different scaling profile. Workflow data is read/written a lot; auth is mostly token checks. Splitting them follows the **single-responsibility principle at the service level** and lets you scale them independently.

Say it like this: "This service is basically CRUD over a `workflows` collection, guarded by the same JWT the auth service issued."

9. Service 3 — Execution Engine

This is the most important service and where most interview questions will go. Take your time here.

Job: actually *run* a workflow — reliably, in the right order, in parallel where safe, with live updates and a saved history.

9.1 Message queue (BullMQ + Redis)

The key design decision. When you call `POST /run-workflow`, the engine does **not** run the workflow during that HTTP request. Instead it:

1. Creates an `executionId`.
2. Pushes a **job** onto a BullMQ queue (stored in Redis).
3. Immediately returns `202 Accepted` with the `executionId`.

```
// producer.ts – enqueue and return fast
await workflowQueue.add("execute", { executionId, userId, nodes, edges }, {
  attempts: 3, // retry up to 3 times
  backoff: { type: "exponential", delay: 2000 } // wait 2s, 4s, 8s between tries
})
```

Why a queue? (memorize these reasons)

- **Durability** — if the engine crashes mid-run, the job is still in Redis and is retried; nothing is lost.
- **Responsiveness** — the API replies in milliseconds instead of blocking for the whole (possibly slow) AI run.
- **Retries & backoff** — transient failures (a flaky API, a rate limit) are retried automatically with growing delays.
- **Back-pressure / concurrency control** — the worker processes a fixed number at a time (concurrency 5), so a traffic spike queues up instead of crashing the box.
- **Horizontal scale** — add more worker processes and they all pull from the same Redis queue.

```
POST /run-workflow → [ Redis Queue ] → Worker #1 (runs 5 at a time)
                    (returns 202)    job, job, job → Worker #2 ← add more to scale
```

Likely Q: "Why not just run it in the request handler?" "Because an AI workflow can take many seconds and can fail. Doing it inline blocks the request, gives no retries, and loses the job if the process dies. A queue gives me durability, retries, and the ability to scale workers independently."

9.2 The worker

A BullMQ **Worker** subscribes to the queue with `concurrency: 5` (five workflows at once). For each job it calls `executeWorkflow()` and is handed the Socket.io server so it can emit live events.

```
new Worker("workflow-execution", async (job) => {
  await executeWorkflow(job.data, io) // io = socket server for live updates
}, { connection: redis, concurrency: 5 })
```

9.3 Topological sort (running nodes in order)

A workflow is a **DAG** (Directed Acyclic Graph) — nodes connected by directed edges, no cycles. Before running, we must order nodes so a node only runs **after** everything feeding into it has run.

We use **Kahn's algorithm**:

1. Compute each node's **in-degree** (how many edges point *into* it).
2. Start with all nodes that have in-degree 0 (no dependencies).
3. Remove one, append it to the order, and decrement the in-degree of its neighbors.
4. Any neighbor that hits in-degree 0 becomes ready.
5. If we processed fewer nodes than exist, there was a **cycle** → throw an error.

```
A → B → D
  |   |
  → C —
Valid order: A, B, C, D (D waits for both B and C)
```

```
function topologicalSort(nodes, edges) {
  const inDegree = new Map(nodes.map(n => [n.id, 0]))
  const adj = new Map(nodes.map(n => [n.id, []]))
  for (const e of edges) { adj.get(e.source).push(e.target); inDegree.set(e.target, inDegree.get(e.target) + 1) }

  const queue = nodes.filter(n => inDegree.get(n.id) === 0).map(n => n.id)
  const order = []
  while (queue.length) {
    const id = queue.shift(); order.push(id)
    for (const next of adj.get(id)) {
      inDegree.set(next, inDegree.get(next) - 1)
      if (inDegree.get(next) === 0) queue.push(next)
    }
  }
  if (order.length !== nodes.length) throw new Error("Workflow contains a cycle")
  return order
}
```

Likely Q: "How do you make sure nodes run in the right order?" "Each workflow is a DAG. I topologically sort it with Kahn's algorithm so every node runs only after its inputs are ready, and the same algorithm detects cycles, which are invalid workflows."

9.4 Node executors (Strategy pattern)

Every node type knows how to execute itself. They all extend an abstract `BaseNode` with one `execute()` method. A **registry** maps a node `type` string to the right executor instance. This is the **Strategy pattern** + a simple **registry/factory**.

```
abstract class BaseNode {
  abstract execute(input: NodeInput): Promise<NodeOutput>
  protected success(data) { return { success: true, data } }
  protected failure(error) { return { success: false, error } }
}

const registry = {
  http: new HttpNode(), llm: new LLMNode(), agent: new AgentNode(),
  condition: new ConditionNode(), webhook: new WebhookNode(), cron: new CronNode()
}
export const getNodeExecutor = (type) => registry[type]
```

The engine loop just does:

```
for (const nodeId of topologicalSort(nodes, edges)) {
  const node = nodes.find(n => n.id === nodeId)
  const executor = getNodeExecutor(node.type)
  const output = await executor.execute({ ...node, previousOutput })
  // emit socket event, store result, pass output forward
}
```

Why this pattern? Adding a new node type = create one class + register it. No `if/else` soup, no touching the engine. **Open/Closed Principle**: open for extension, closed for modification.

The 6 node types:

- **HTTP** — `axios` request to a URL, returns the response data.
- **LLM** — sends a prompt to a chosen model via OpenRouter.
- **Agent** — the ReAct loop (below).
- **Condition** — safely evaluates a boolean expression to branch.
- **Webhook** — receives an external trigger payload.
- **Cron** — represents a schedule (`0 9 * * *` = every day 9am).

Data passing: a node can reference the previous node's result with `{{previous.output}}`, which the engine interpolates before running.

9.5 The ReAct AI agent

The **Agent** node implements the **ReAct** pattern = **Reason + Act**. Instead of one LLM call, it loops: the model *thinks*, optionally *calls a tool*, sees the result, and repeats — up to 5 steps — until it produces a final answer.

```
Question → LLM: "Thought + Action"
      |
      |— Action = use a tool? → run tool (calculator/weather) → feed result back —┐
      |                                                                    |
      |— Action = final answer? → return answer                                (loop, max 5)←┘
```

```
for (let step = 0; step < 5; step++) {
  const response = await callLLM(prompt + context, model) // model returns {thought, action, input}
  if (response.action === "final") return response.input
  const toolResult = runTool(response.action, response.input) // e.g. calculator
  context += `\nObservation: ${toolResult}` // feed back, loop again
}
```

Tools available: a safe `calculator` and a mock `weather` tool. The loop cap (5) prevents infinite loops / runaway cost.

Likely Q: "What is an AI agent vs. just calling an LLM?" "A plain LLM call is one-shot. An agent runs a loop: it reasons about what to do next, can call tools to get real data or do exact math, observes the result, and continues until it's confident. That's the ReAct pattern — Reason then Act."

9.6 Multi-model LLM support

All AI calls go through one function, `callLLM(prompt, model, systemPrompt)`, which routes to **OpenRouter**. OpenRouter exposes OpenAI, Anthropic, and Groq behind a single API + key, so swapping models is just changing a string.

```
const MODELS = {
  "openai/gpt-3.5-turbo": { baseUrl: "https://openrouter.ai/api/v1" },
  "openai/gpt-4o": { baseUrl: "https://openrouter.ai/api/v1" },
  "anthropic/claude-3-5-sonnet": { baseUrl: "https://openrouter.ai/api/v1" },
  "groq/llama-3-70b-versatile": { baseUrl: "https://openrouter.ai/api/v1" }
}
```

Why one interface? Each node can pick its model. Cheap/fast model for simple steps, a stronger model for hard reasoning — without changing any other code.

9.7 Real-time updates (Socket.io)

So the canvas can light up live, the engine emits WebSocket events as it runs.

How rooms work:

1. The client gets an `executionId` from `/run-workflow`.
2. It connects via Socket.io and emits `join_execution` with that id → the server puts that socket into a **room** named after the `executionId`.
3. As the worker runs, it emits to that room only: `node_started`, `node_completed`, `node_failed`, `workflow_completed`.
4. The client updates each node's color from grey → blue (running) → green/red.

```
// server: socket joins a room per execution
socket.on("join_execution", (id) => socket.join(id))

// worker: emit only to the right room
io.to(executionId).emit("node_completed", { nodeId, output })
```

Why rooms? So a user only receives events for *their* run, not everyone's. It scopes the broadcast.

Likely Q: "Why WebSockets and not polling?" "Execution status changes second-by-second. Polling wastes requests and lags. A WebSocket pushes the moment something happens — instant and efficient."

9.8 Execution logging

Every run is saved as an `ExecutionLog` document: `executionId, userId, status, nodeResults[]` (per-node `status/output/error/duration`), `startedAt, completedAt, totalDurationMs, trigger`. This powers the run-history view and debugging.

10. The frontend

Next.js 14 App Router with these pieces:

- **React Flow** renders the canvas — draggable nodes, connectable edges, minimap, controls.
- **Zustand** is the global store: `nodes, edges, selectedNode, nodeStates (live status), executionResults, UI flags`. It's like a tiny Redux without the boilerplate.
- **React Query** handles server data (list/save workflows) with caching and auto-refetch.
- **Socket.io-client** subscribes to live execution events and writes node statuses into Zustand, which re-renders the nodes.
- **Tailwind CSS** for styling; **Framer Motion** for the log drawer animation; **sonner** for toasts.
- **Runtime config** — the app reads backend URLs from `window.__ENV__` (injected at container start) so the *same* build works against any deployment URLs.

Pages: `/` landing page, `/login`, `/signup`, `/app` (the protected canvas).

```
// Zustand store shape (simplified)
{
  nodes, edges, selectedNode,
  nodeStates: { [nodeId]: { status, output, error } }, // drives live colors
  isRunning, executionId,
  addNode, setEdges, setNodeState, ...
}
```

Likely Q: "Why Zustand over Redux?" "For this app's needs Redux is overkill. Zustand gives global state with a tiny API and no boilerplate, and React Flow's frequent updates stay fast."

11. End-to-end flow

What happens when you press "Run" — the whole story:

1. Browser —POST /run-workflow {nodes,edges} + JWT→ Execution Engine
2. Engine verifies JWT, makes executionId, enqueues job in Redis, returns 202 {executionId}
3. Browser opens Socket.io, emits join_execution(executionId) → joins that room
4. Worker pulls job → topological sort → for each node:
 - emit node_started → run executor → emit node_completed/failed
5. Browser receives each event → updates node color + output live
6. Worker finishes → saves ExecutionLog to MongoDB → emit workflow_completed
7. Browser shows toast + full results in the log drawer

If you can narrate these 7 steps, you can answer almost any "how does it work" question.

12. Data models

```
// User (auth-service)
{ email: string, password: string /* bcrypt hash */, createdAt }
```

```
// Workflow (workflow-service)
{ userId, name, description, nodes: WorkflowNode[], edges: WorkflowEdge[],
  isActive: boolean, tags: string[], timestamps }
```

```
// WorkflowNode (shared-types)
{ id: string, type: "http"|"llm"|"agent"|"condition"|"webhook"|"cron",
  data: { label, query?, url?, model?, condition?, ... }, position: {x, y} }
```

```
// ExecutionLog (execution-engine)
{ executionId, userId, status: "running"|"completed"|"failed",
  nodeResults: [{ nodeId, nodeType, status, output, error, durationMs }],
  startedAt, completedAt, totalDurationMs, trigger }
```

Why MongoDB? Workflows are variable-shaped JSON (different nodes have different fields). A document database stores that naturally without rigid schemas or joins.

13. Security

Concern	How it's handled
Passwords	bcrypt , 12 rounds, salted — never stored in plaintext.
Identity	JWT signed with a secret, 7-day expiry, verified on every protected route.
HTTP headers	helmet sets secure defaults (XSS, sniffing, etc.).
Abuse	express-rate-limit caps requests per IP.
Cross-origin	CORS restricts which origins may call the APIs.
Code injection	The Condition node sandboxes expressions — it blocks <code>require</code> , <code>process</code> , <code>exec</code> , <code>spawn</code> , <code>global</code> , <code>eval</code> before evaluating, so users can't run arbitrary code.
Secrets	API keys & DB URIs are environment variables , never committed.

Likely Q: "The condition node runs user expressions — isn't that dangerous?" "Yes, that's why it's sandboxed. Before evaluating I reject any expression containing dangerous keywords like `require`, `process`, `child_process`, or `eval`, so only simple comparisons like `{{previous.output}} > 100` are allowed."

14. DevOps

- **Docker (multi-stage builds)** — each service has a Dockerfile with `deps` → `build` → `runner` stages, so the final image is small and contains only what it needs.
- **docker-compose** — one command (`docker compose up`) starts the whole stack: MongoDB, Redis, all 3 services, the frontend, plus Prometheus & Grafana.
- **GitHub Actions CI** — on every push: `lint` → `test` → `build` → `docker build`, with MongoDB and Redis service containers for integration tests.
- **Prometheus + Grafana** — Prometheus scrapes metrics from the services; Grafana visualizes them.
- **Cloud deploy** — a `render.yaml` blueprint provisions all services + a Redis instance; MongoDB is hosted on Atlas. The frontend reads backend URLs at runtime so one image works anywhere.

Likely Q: "What does a multi-stage Docker build buy you?" "The build stage has all the dev dependencies and compiles the code; the final runner stage copies only the built output and production deps. Smaller, faster, more secure images."

15. Testing

Vitest, ~29 tests across unit + integration:

- `topologicalSort.test.ts` — linear chains, parallel branches, **cycle detection**.

- `calculator.test.ts` — math correctness **and** rejecting dangerous input.
- `conditionNode.test.ts` — true/false eval, interpolation, **blocking dangerous keywords**.
- `httpNode.test.ts` — mocked `axios` success and failure paths.
- `agentExecutor.test.ts` — mocked LLM: tool use, final answer, max-steps, unknown tool.
- `auth.test.ts` — integration tests for signup/login with mocked Mongoose & bcrypt.

Say it like this: "I focused tests on the risky logic — the graph sort, the agent loop, and the security sandboxes — not on trivial getters."

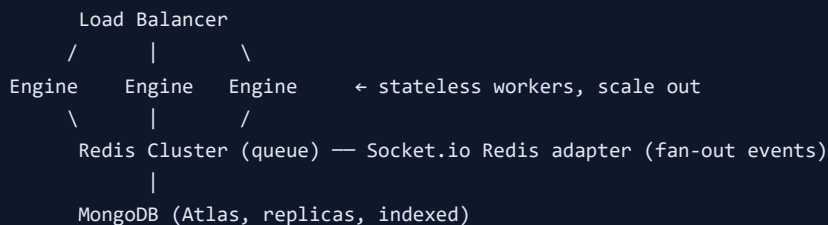
16. System design & scaling

"How would you scale this to 10,000 concurrent workflows?" (a classic) — answer in layers:

1. **Workers** — the execution worker is stateless; run many replicas, all pulling from the same Redis queue. Scale horizontally.
2. **Redis** — move to Redis Cluster / managed Redis for queue throughput.
3. **MongoDB** — use Atlas with replicas; index `userId` and `executionId`.
4. **WebSockets** — add the **Socket.io Redis adapter** so events broadcast correctly across multiple engine instances.
5. **Auth** — stateless JWT means the auth service scales trivially behind a load balancer; cache nothing or cache the public key.
6. **Rate limiting** — per-user limits to protect shared AI quotas.
7. **Cost control** — cap agent steps, choose cheaper models for simple nodes, add caching for identical LLM prompts.

Bottlenecks to name: the LLM API (slowest + rate-limited), Redis throughput, and MongoDB writes for logs.

Mitigations: queue concurrency tuning, batching, and archiving old logs.



17. Trade-offs & "what would you improve?"

Being honest here signals maturity. Pick a few:

- **More node types** — database, email, Slack, branching/merge nodes.
- **Persisted cron scheduling** — currently cron is represented but a production scheduler (e.g. a dedicated scheduler service or BullMQ repeatable jobs) would actually fire them.
- **Per-user secrets vault** — so users store their own API keys securely.
- **Streaming LLM output** token-by-token to the UI.
- **Observability** — wire real metrics/traces (OpenTelemetry) and alerting.
- **Auth hardening** — refresh tokens, email verification, OAuth.
- **Tests** — add end-to-end (Playwright) and load tests (k6).

Say it like this: "It's a strong foundation. The biggest next steps would be a real cron scheduler, a per-user secrets vault, and token-streaming for LLM responses."

18. Interview Q&A bank

Basics

Q: What is AgentFlow in one line? A visual, drag-and-drop platform to build and run AI-powered automation workflows, with a microservice backend and a job queue.

Q: What are the main parts? A Next.js frontend, three Express microservices (auth, workflow, execution engine), MongoDB, and a Redis/BullMQ queue, tied together with Socket.io for real-time updates.

Q: What's a "node" and an "edge"? A node is one step (HTTP call, LLM prompt, etc.); an edge is a connection that defines order and data flow between two nodes.

Intermediate

Q: Walk me through running a workflow. *(Use the 7-step end-to-end flow from section 11.)*

Q: Why a message queue? Durability (survive crashes), responsiveness (return immediately), retries with backoff, concurrency control, and horizontal scaling of workers.

Q: How do nodes run in the correct order? Topological sort (Kahn's algorithm) over the workflow DAG; it also detects invalid cyclic workflows.

Q: How does data flow between nodes? Each node's output is passed forward; a node can reference the prior result with `{{previous.output}}`, which the engine interpolates before execution.

Q: How do live updates reach the browser? Socket.io. The client joins a room named by `executionId`; the worker emits per-node events to that room; the UI updates node colors.

Q: How is the code organized to add a new node type? Strategy pattern: extend `BaseNode`, implement `execute()`, register it in the registry. No engine changes — Open/Closed Principle.

Advanced

Q: What happens if a worker crashes mid-execution? The job stays in Redis. BullMQ detects the stalled job and re-queues it; with `attempts: 3` and exponential backoff it retries. The execution log reflects the final state.

Q: How would you prevent duplicate or double execution? Use the `executionId` as an idempotency key and BullMQ job IDs; make node side-effects idempotent or guard with a "already processed" check.

Q: How do you scale WebSockets across multiple engine instances? Use the Socket.io Redis adapter so an event emitted on instance A reaches a client connected to instance B.

Q: What's a ReAct agent and why use it? Reason + Act loop: the model thinks, optionally calls a tool, observes the result, and repeats until it answers. It's better than a single call for tasks needing real data or exact computation. We cap it at 5 steps to bound cost.

Q: How do you support multiple AI models cleanly? A single `callLLM()` interface routes to OpenRouter, which fronts many providers. Switching models is a config string, so each node can pick its model.

Q: Biggest performance bottleneck? The LLM API — it's the slowest and is rate-limited. Mitigate with concurrency tuning, cheaper models for simple steps, prompt caching, and backoff on 429s.

Project / behavioral

Q: What was the hardest part? Designing the execution engine — getting ordering (topological sort), reliability (queue + retries), and live updates (Socket.io rooms) to work together cleanly.

Q: What did you learn? How to design around a queue for reliability, how the Strategy pattern keeps an engine extensible, and the realities of deploying a multi-service app (CORS, runtime config, container builds).

Q: What would you do differently? Add a real cron scheduler and a per-user secrets vault earlier; write end-to-end tests sooner.

19. Glossary

- **API** — a way for programs to talk to each other over HTTP.
 - **JWT (JSON Web Token)** — a signed token proving who you are; stateless auth.
 - **bcrypt** — a slow, salted password-hashing function.
 - **Microservice** — a small, independent service doing one job.
 - **Message queue** — a buffer that holds jobs so workers can process them reliably/asynchronously.
 - **BullMQ** — a Node.js queue library built on Redis.
 - **Redis** — a fast in-memory data store (used here for the queue).
 - **DAG** — Directed Acyclic Graph: nodes + directed edges, no cycles.
 - **Topological sort** — ordering a DAG so dependencies come first.
 - **Kahn's algorithm** — a specific way to do a topological sort using in-degrees.
 - **Strategy pattern** — interchangeable classes behind one interface (our node executors).
 - **ReAct agent** — an LLM loop that Reasons then Acts (uses tools) repeatedly.
 - **WebSocket / Socket.io** — a persistent two-way connection for live updates.
 - **Room (Socket.io)** — a named group of sockets you can broadcast to.
 - **Idempotent** — doing it twice has the same effect as once.
 - **Horizontal scaling** — adding more machines/instances rather than a bigger one.
 - **Multi-stage Docker build** — separate build vs runtime stages for small images.
 - **CI/CD** — automated testing/building/deploying on every change.
-

20. One-page cheat sheet

```
WHAT:    Visual AI workflow automation (n8n + Zapier for AI agents)
STACK:   Next.js · TS · Express · MongoDB · Redis/BullMQ · Socket.io · Docker
SHAPE:   3 microservices (Auth 4001, Workflow 4002, Engine 4003) in a Turborepo
NODES:   HTTP · LLM · Agent · Condition · Webhook · Cron (Strategy pattern + registry)
RUN:     POST /run-workflow → enqueue (Redis) → worker → topo-sort → run nodes
        → Socket.io live events → save ExecutionLog (MongoDB)
QUEUE:   why = durability, retries+backoff, fast API, concurrency, scale-out
ORDER:   DAG + Kahn's topological sort (also detects cycles)
AGENT:   ReAct loop (reason→act→observe), max 5 steps, tools = calculator/weather
AI:      one callLLM() → OpenRouter → GPT-4o / Claude 3.5 / Groq Llama
LIVE:    client joins room by executionId; worker emits node_started/completed/failed
SECURE:  bcrypt · JWT · helmet · rate-limit · CORS · expression sandbox
DEVOPS:  multi-stage Docker · docker-compose · GitHub Actions · Prometheus/Grafana
SCALE:   stateless workers ×N · Redis cluster · Socket.io Redis adapter · Mongo replicas
```

Final advice

- Practice the **2-minute walkthrough** and the **7-step run flow** out loud until they're automatic.
- For any feature, be ready to answer "**why did you build it that way?**" — the *why* impresses more than the *what*.
- It's fine to say "**I'd improve X**" — it shows you understand trade-offs.
- If you don't know something, say how you'd find out. Honesty + reasoning beats bluffing.

You built something real and deployed it. Now own it. Good luck! 🚀